

Quadratic Optimal Regulator (LQR)

Letting a cost function choose the feedback gain for you.

Ogata, *Modern Control Engineering*, Ch. 10.

In this tutorial you will:

- minimize $J = \int (x^T Q x + u^T R u) dt$ with `lqr` and the Riccati equation,
- see how the Q/R weights trade response speed against control effort, and
- verify LQR's guaranteed phase margin with `margin`.

Step through with **Ctrl+Enter**, or render a report with `publish`.

Rather than placing poles at hand-chosen locations, the **linear quadratic regulator** (LQR) chooses the feedback gain K in $u = -Kx$ to minimize a performance index trading off state deviation against control effort:

$$J = \int_0^\infty (x^T Q x + u^T R u) dt$$

where $Q \geq 0$ weights state error and $R > 0$ weights control effort. The optimal gain is $K = R^{-1} B^T P$, where $P = P^T \geq 0$ solves the algebraic Riccati equation (ARE):

$$A^T P + P A - P B R^{-1} B^T P + Q = 0$$

Contents

- Plant
- Solving the Riccati Equation
- Before vs. After: Open-Loop vs. LQR-Regulated Response
- Effect of Q and R on the Tradeoff
- Guaranteed Stability and Robustness Margins
- Try it yourself

Plant

```
A = [0 1; -1 -1];  
B = [0; 1];  
C = [1 0];  
D = 0;
```

Solving the Riccati Equation

`'care'` solves the continuous-time ARE directly; `'lqr'` wraps this and additionally returns the optimal gain K and closed-loop eigenvalues.

```
Q = diag([10 1]);  
R = 1;  
[K,P,clp] = lqr(A,B,Q,R);  
fprintf('LQR gain K = '); disp(K)  
fprintf('Riccati solution P = \n'); disp(P)  
fprintf('Closed-loop poles = '); disp(clp')
```

```
LQR gain K =      2.3166      1.5755
```

```
Riccati solution P =  
    7.5420    2.3166  
    2.3166    1.5755
```

```
Closed-loop poles =   -1.2878 - 1.2878i   -1.2878 + 1.2878i
```

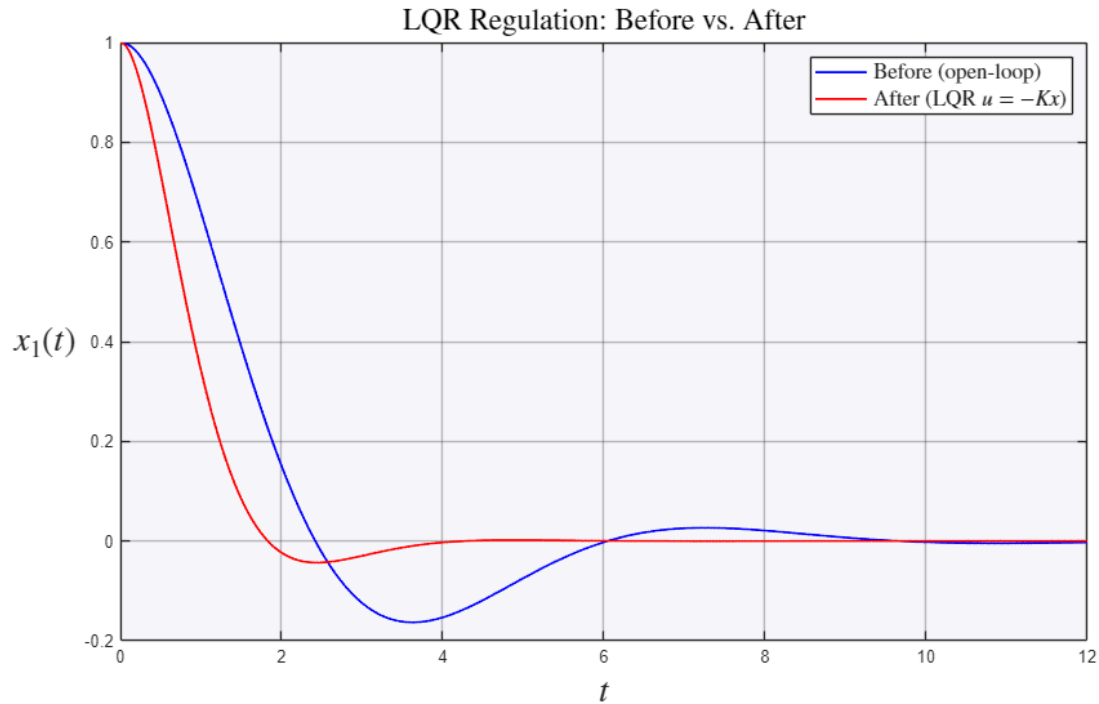
Cross-check via ‘care’ directly: $A^T P + PA - PBR^{-1}B^T P + Q = 0$.

```
[P_care,~,K_care] = care(A,B,Q,R);  
fprintf('\ncare() gain (= R^{-1}B^TP, sign convention matches lqr K): ')  
disp(K_care)  
residual = A'*P_care + P_care*A - P_care*B*(R\B')*P_care + Q;  
fprintf('||Riccati residual|| = %.2e (should be ~0)\n', norm(residual))  
  
care() gain (= R^{-1}B^TP, sign convention matches lqr K):      2.3166      1.5755  
  
||Riccati residual|| = 1.16e-14 (should be ~0)
```

Before vs. After: Open-Loop vs. LQR-Regulated Response

From the same initial state, the open-loop plant rings (lightly damped complex poles); the LQR feedback $u=-Kx$ pulls the state to zero quickly and smoothly – the “after” of optimal regulation.

```
x0 = [1; 0];  
t = 0:0.01:12;  
x_ol = initial(ss(A,zeros(2,1),eye(2),0), x0, t);  
x_lqr = initial(ss(A-B*K,zeros(2,1),eye(2),0), x0, t);  
figure  
plot(t,x_ol(:,1),'b','LineWidth',1.3)  
hold on  
plot(t,x_lqr(:,1),'r','LineWidth',1.3)  
hold off  
grid on  
legend('Before (open-loop)','After (LQR  $u=-Kx$ )','Interpreter','latex','FontSize',13)  
title('LQR Regulation: Before vs. After','Interpreter','latex','FontSize',17)  
ylabel('$x_1(t)$','Interpreter','latex','FontSize',20)  
set(get(gca, 'YLabel'), 'Rotation', 0)  
xlabel('$t$','Interpreter','latex','FontSize',20)
```



Effect of Q and R on the Tradeoff

Increasing Q (penalize state error more) drives faster, more aggressive poles; increasing R (penalize control effort more) yields gentler, slower poles and smaller gains. This is the central LQR design knob, replacing the trial-and-error of direct pole placement.

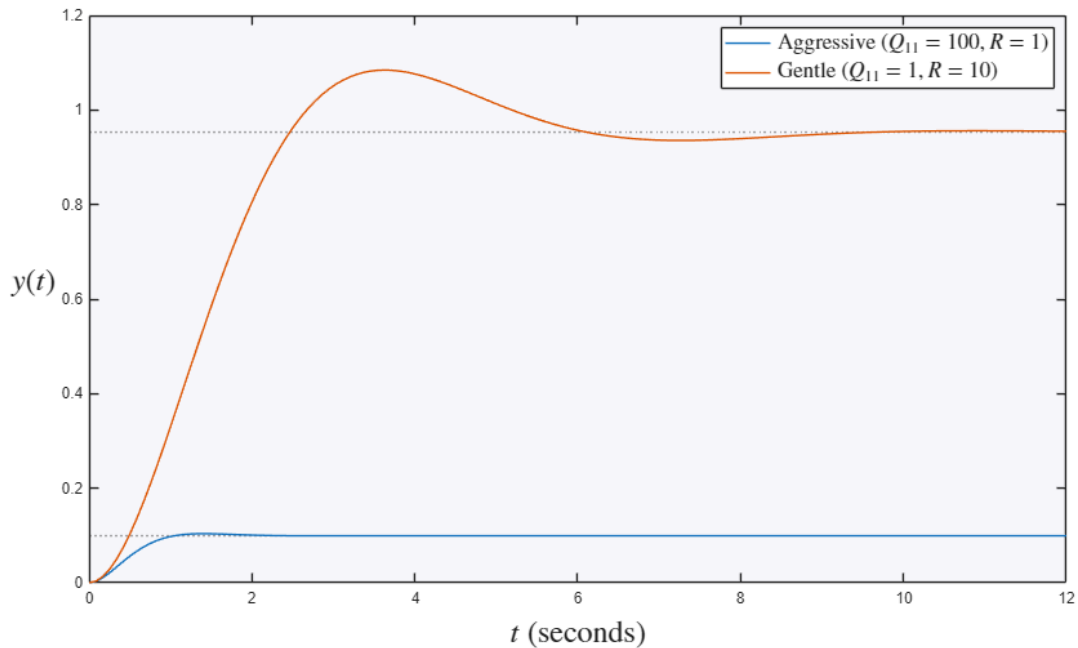
```
[K_aggressive,~,~] = lqr(A,B,diag([100 1]),1);
[K_gentle,~,~]      = lqr(A,B,diag([1 1]),10);
fprintf('\nAggressive (Q11=100,R=1): K = '); disp(K_aggressive)
fprintf('Gentle (Q11=1,R=10):      K = '); disp(K_gentle)

sys_aggr = ss(A-B*K_aggressive,B,C,D);
sys_gentle = ss(A-B*K_gentle,B,C,D);
figure
hold on
step(sys_aggr)
step(sys_gentle)
hold off
legend('Aggressive ($Q_{11}=100,R=1$)', 'Gentle ($Q_{11}=1,R=10$)', ...
      'Interpreter','latex','FontSize',14)
title('LQR: Effect of $Q/R$ Weighting on Step Response','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)

Aggressive (Q11=100,R=1): K =      9.0499      3.4833

Gentle (Q11=1,R=10):      K =      0.0488      0.0944
```

LQR: Effect of Q/R Weighting on Step Response



Guaranteed Stability and Robustness Margins

A classical result: full-state-feedback LQR (with R scalar) always achieves at least 60-degree phase margin and infinite gain margin (gain reduction tolerance) at the plant input – a robustness guarantee that generic pole placement does not provide.

```
sys_ol_lqr = ss(A,B,K,0);  
[~,PM_lqr] = margin(sys_ol_lqr);  
fprintf('\nLQR loop-gain phase margin at plant input = %.2f deg (>=60 guaranteed)\n', PM_lqr)
```

```
LQR loop-gain phase margin at plant input = 86.55 deg (>=60 guaranteed)
```

Try it yourself

- Crank $Q(1,1)$ up to 500 and notice the poles get faster and K grow – you are buying speed with control effort.
- Raise R instead and watch the response get gentle and the gain shrink.